

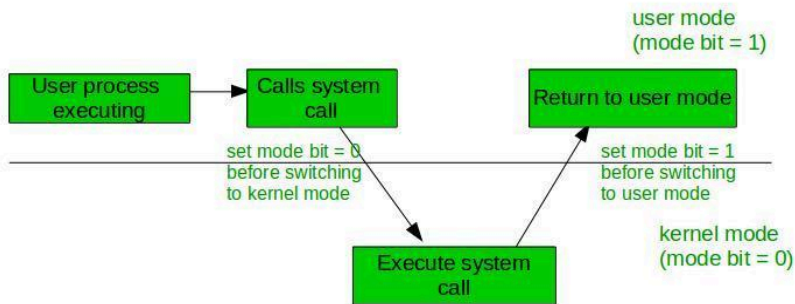
**Department of Computer Science and Engineering****Program: BE**

Date		Maximum Marks	50
Course Code	CS235AI	Duration	90Min
Sem	III	CIE – I	
OPERATING SYSTEMS (Common to CSE/CD/CY/ISE)			

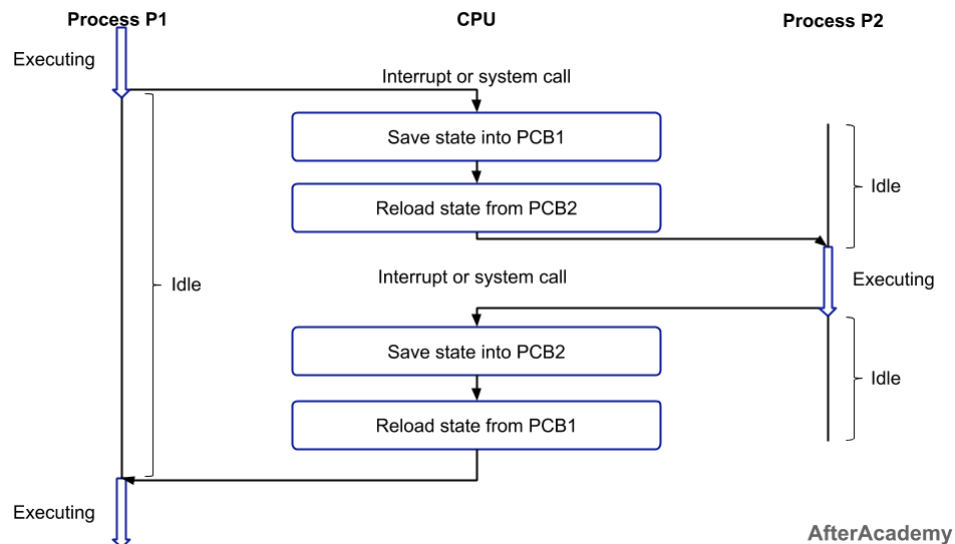
Sl. No.	Quiz Questions	M
1	<i>Identify the multithreading model where</i> <i>i. The entire process blocks when a thread makes blocking system call</i> <i>ii. The entire process does NOT blocks when a thread makes blocking system call</i> i. Many-to-One ii. One-to-One	02
2	<i>Analyze the following code segment and predict the output.</i> <code>#include <stdio.h> #include <unistd.h> #include <sys/wait.h> int main() { printf("A"); pid_t p1 = fork(); if (p1 == 0) { printf("B"); fork(); printf("C"); exit(0); } else { wait(NULL); printf("D"); } printf("E"); return 0; }</code>	02

	Possible multiple valid sequences such as: A B C C D E , A B C D C E , A B C D E C	
3	<i>For each of the following scenarios, identify the current process state and the state transition that will occur next:</i> i. A process is executing on the CPU when a higher-priority process becomes ready. ii. A process is waiting for an I/O operation and the I/O request completes. i. Running → Ready ii. Blocked → Ready	02
4	<i>Categorize the following operations as Privileged or Unprivileged:</i> i. Changing the address space of a running process ii. Spawning a child process using fork() iii. Reading the thread ID of the current thread iii. Executing a context switch between two different processes i. Privileged ii. Privileged iii. Unprivileged iv. Privileged	02
5	<i>Identify the memory segment where each variable and string literal is stored during program execution.</i> <pre>char msg[] = "OS Quiz"; char *ptr = "Memory"; int main() { register int r = 10; int *arr = malloc(sizeof(int) * 3); }</pre> msg → Data ptr → Data "Memory" → Text r → Stack arr → Stack malloc memory → Heap	02

Sl. No.	Test Questions	M
1a	Write a multithreaded program in C to perform concurrent operations on an array using two threads. Thread-1 Counting even and odd numbers while Thread-2 Calculating the average of array elements. Execute both tasks simultaneously and display the results. <pre> 5+5=10M #include <stdio.h> #include <pthread.h> int arr[] = {5, 2, 8, 7, 10, 3}; int n = 6; int even = 0, odd = 0; float average = 0.0; void* countEvenOdd(void* arg) { for (int i = 0; i < n; i++) { if (arr[i] % 2 == 0) even++; else odd++; } pthread_exit(NULL); } void* findAverage(void* arg) { int sum = 0; for (int i = 0; i < n; i++) sum += arr[i]; average = (float)sum / n; pthread_exit(NULL); } int main() { pthread_t t1, t2; pthread_create(&t1, NULL, countEvenOdd, NULL); pthread_create(&t2, NULL, findAverage, NULL); pthread_join(t1, NULL); </pre>	10

	<pre>pthread_join(t2, NULL); printf("Even Numbers: %d\n", even); printf("Odd Numbers: %d\n", odd); printf("Average: %.2f\n", average); return 0; }</pre>	
2a	A user application needs to access a printer and allocate memory while running. If applications could directly interact with hardware or use privileged instructions, they might corrupt memory, crash the system, or compromise security. From this scenario, identify and explain the operating system mechanism that prevents such direct access to critical resources. How does it work, and what could happen if this protection were absent? Include a neat diagram showing the transitions involved. Diagram(2M)+explantion(4M)=6M Mechanism: Dual-Mode Operation (User Mode & Kernel Mode) Purpose: Protect hardware and critical resources from unauthorized access. Working: • User programs run in User Mode • Privileged operations (I/O, memory mgmt.) require system calls → switch to Kernel Mode • OS validates requests and executes safely • If absent: Risks include system crash, memory corruption, security breaches  <pre> graph TD A[User process executing] -- "set mode bit = 0 before switching to kernel mode" --> B[Calls system call] B --> C[Execute system call] C -- "set mode bit = 1 before switching to user mode" --> D[Return to user mode] D --> A </pre> The diagram illustrates the transition between User Mode and Kernel Mode. It starts with a box labeled 'User process executing'. An arrow points to a box labeled 'Calls system call', with the label 'set mode bit = 0 before switching to kernel mode' written below it. From 'Calls system call', an arrow points down to a box labeled 'Execute system call'. From 'Execute system call', an arrow points up to a box labeled 'Return to user mode', with the label 'set mode bit = 1 before switching to user mode' written above it. Finally, an arrow points from 'Return to user mode' back to 'User process executing'. To the right of the diagram, there are two labels: 'user mode (mode bit = 1)' at the top and 'kernel mode (mode bit = 0)' at the bottom.	06
2b	<i>With neat diagram discuss the mechanism of context switch between processes in operating system.</i> Diagram 2M + explanation 2M= 4M	04

A **context switch** is the process by which the operating system saves the current state (CPU registers, program counter, stack pointer, PCB) of a running process and restores the saved state of another ready process to resume its execution. It is handled by the **kernel scheduler**, allowing multitasking by rapidly switching CPU control among multiple processes.



3a A data analytics system uses process creation to speed up computation. Write a C program where the parent process collects system data and then creates two child processes using `fork()`. Child Process 1 should analyze the dataset to determine and display the highest CPU usage recorded, while Child Process 2 should calculate and display the average CPU usage. Each child must print its PID, perform its computation, display the results, and then exit. The parent process should wait for both children to finish, print a confirmation message indicating their termination, and finally display "Parent Process Exiting...".

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main() {
    int usage[] = {40, 55, 70, 65, 90}; // Example CPU usage dataset
    int n = 5;
    pid_t p1, p2;

    p1 = fork();
    if (p1 == 0) {
        // Child Process 1: Max CPU usage
```

10

	<pre> int max = usage[0]; for (int i = 1; i < n; i++) if (usage[i] > max) max = usage[i]; printf("Child-1 PID: %d → Max CPU Usage: %d%%\n", getpid(), max); return 0; } p2 = fork(); if (p2 == 0) { // Child Process 2: Average CPU usage int sum = 0; for (int i = 0; i < n; i++) sum += usage[i]; printf("Child-2 PID: %d → Average CPU Usage: %.2f%%\n", getpid(), (float)sum / n); return 0; } // Parent waits for both children wait(NULL); wait(NULL); printf("Both child processes terminated.\n"); printf("Parent Process Exiting...\n"); return 0; } </pre>	
4a	<i>Brief Amdahl's Law in the context of multi threaded programming. Given that 60% of a program's code can be parallelized and 40% must be executed sequentially, calculate the theoretical speedup when utilizing 4 threads/cores for parallel execution. Is Amdahl's Law practical for real-world computing? Justify your answer.</i> 2+2+2=6M	06

	◆ Given Data • $S = 0.40$ (40% serial) • $1 - S = 0.60$ (60% parallel) • $N = 4$ (4 threads/cores) ◆ Substitute in the formula $\text{Speedup} = \frac{1}{0.40 + \frac{0.60}{4}}$ Step-by-step: $\frac{0.60}{4} = 0.15$ $0.40 + 0.15 = 0.55$ $\text{Speedup} = \frac{1}{0.55} = 1.82$ Amdahl's Law is a useful theoretical guideline, but in real systems overheads (e.g., synchronization, communication) reduce actual gains, making it somewhat optimistic though still practically informative.	
4b	<i>Justify or contradict the following statements:</i> <i>The primary objective of multiprogramming is to minimize user response time, while the primary objective of time sharing is to maximize processor utilization.</i> <i>A microkernel-based operating system is more efficient, reliable, secure, and flexible than a monolithic</i> 2+2=4M i. Contradict Multiprogramming primarily aims to maximize CPU utilization by keeping multiple processes in memory. Time-sharing focuses on minimizing response time and providing fast user interaction. ii. Justify Microkernels are more reliable, secure, and flexible because only essential services run in kernel mode.	04
5a	<i>Explain the various kernel architectures used in operating systems and explain how their components are organized and interact with each other.</i>	10

	2.5*4=10M Operating systems use different kernel architectures to organize and manage system components such as process scheduling, memory management, I/O, and device drivers: • Monolithic Kernel: All OS services (file system, device drivers, memory, scheduling) run in kernel mode as a single large module. It offers high performance but has low fault isolation since errors anywhere can crash the system. • Microkernel: Only essential functions like IPC, basic scheduling, and memory management run in kernel mode, while other services run in user mode. This improves reliability, security, and modularity, but may reduce performance due to increased message passing. • Hybrid Kernel: Combines monolithic performance with microkernel modularity. Some services run in kernel mode to improve speed while others remain in user mode for stability. Examples: Windows, macOS. • Modules (Loadable Kernel Modules): Allows drivers and services to be loaded/unloaded dynamically into kernel space, improving flexibility and maintainability. Used in Linux.	
--	--	--

Course Outcomes

CO 1	Demonstrate the fundamental concepts of operating system like process management, file management, memory management and issues of synchronization.
CO 2	Analyze and interpret operating system concepts to acquire a detailed understanding of the course.
CO 3	Apply the operating systems concepts to address related new problems in computer science Domain.
CO 4	Design or develop solutions to solve applicable problems in operating systems domain.
CO5	Extend the theoretical knowledge acquired through the course to demonstrate skills like investigation, effective communication, working in team/Individual, following ethical practices by implementing operating system concepts/applications and engage in lifelong learning.

Blooms' taxonomy

L1	L2	L3	L4	L5	L6	CO1	CO2	CO3	CO4	CO5	CO6
02	28	28	2			10	20	14	16		